

Assignment no. 3:

Practical Assignments on Data Visualization and Exploratory Data Analysis (EDA)

Objective: To develop skills in data visualization using libraries like Matplotlib and Seaborn and to perform Exploratory Data Analysis (EDA) using summary statistics, data distribution analysis, and correlation analysis.

3.1 Data Visualization Libraries

Data visualization libraries convert raw numeric data and complex datasets into clear, interactive graphs, charts, maps, and dashboards.

1. Basic Data Visualization with Matplotlib:

Matplotlib is the foundational charting library for Python. It uses a hierarchical structure where a Figure acts as the overall window or canvas, and Axes represent the individual plots or subplots containing the data, labels, and gridlines.

Use Matplotlib to create a simple line plot, bar chart, and scatter plot using a sample dataset.

[1]:

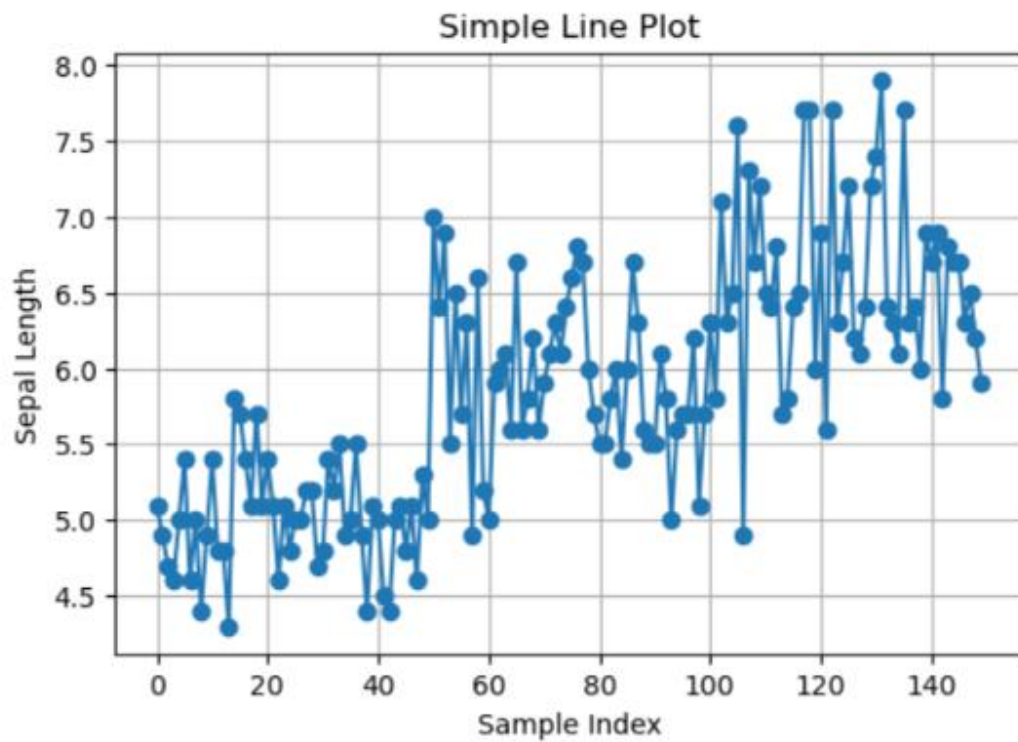
```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("iris")
print(df.head())
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

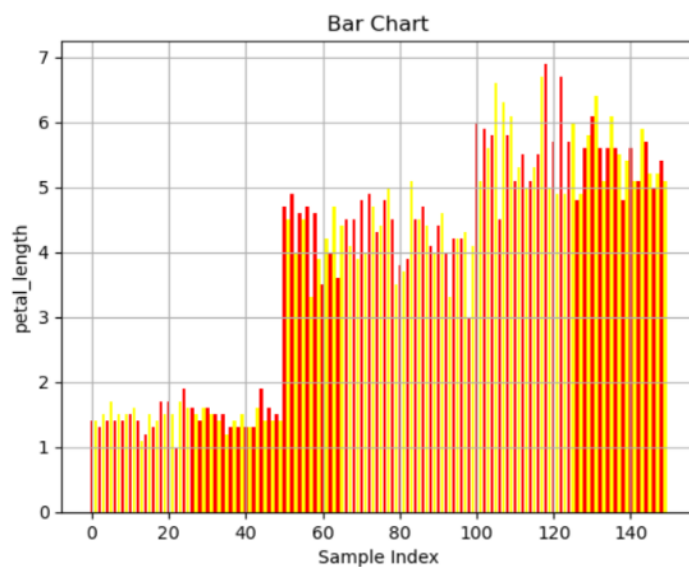
[5]:

```
plt.figure(figsize=(6,4))
plt.plot(df.index, df["sepal_length"], marker="o", linestyle="-")
plt.xlabel("Sample Index")
plt.ylabel("Sepal Length")
plt.title("Simple Line Plot")
plt.grid(True)
plt.show()
```



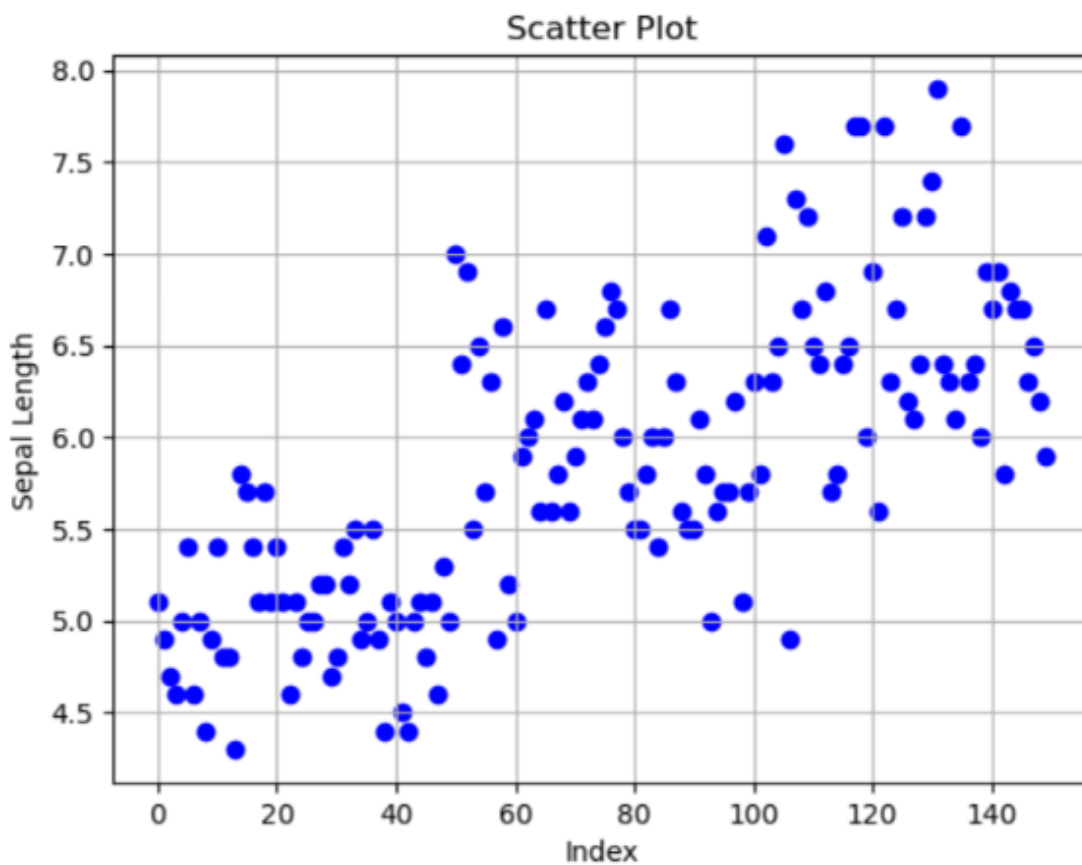
[7]:

```
plt.bar(df.index,df["petal_length"],color=["red","yellow","red","yellow"])
plt.title("Bar Chart")
plt.xlabel("Sample Index")
plt.ylabel("petal_length")
plt.grid()
plt.show()
```



[8]:

```
plt.scatter(df.index,df["sepal_length"],color="b",marker="o")  
plt.title("Scatter Plot")  
plt.xlabel("Index")  
plt.ylabel("Sepal Length")  
plt.grid()  
plt.show()
```



2. Advanced Data Visualization with Seaborn:

Seaborn is a Python library for creating attractive statistical visualizations. Built on Matplotlib and integrated with Pandas, it simplifies complex plots like line charts, heatmaps and violin plots with minimal code.

Use Seaborn to create a histogram, box plot, and heatmap using a sample dataset.

[11]:

```
ds=sns.load_dataset("penguins")
print(ds.head())
print(ds.shape)
```

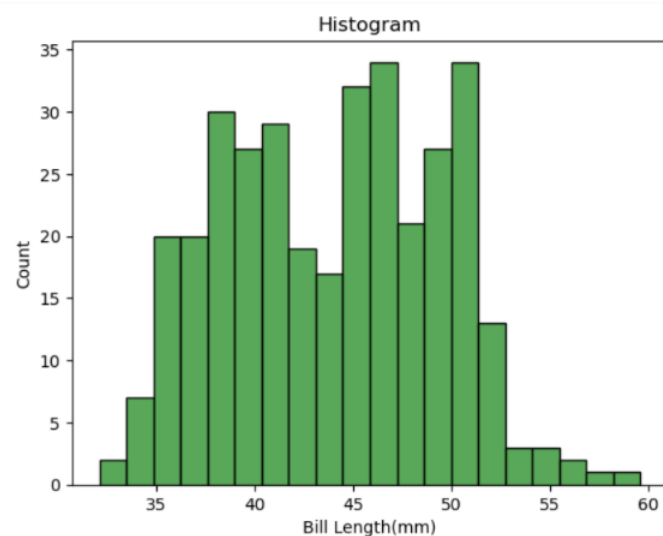
	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	
2	Adelie	Torgersen	40.3	18.0	195.0	
3	Adelie	Torgersen	NaN	NaN	NaN	
4	Adelie	Torgersen	36.7	19.3	193.0	

	body_mass_g	sex
0	3750.0	Male
1	3800.0	Female
2	3250.0	Female
3	NaN	NaN
4	3450.0	Female

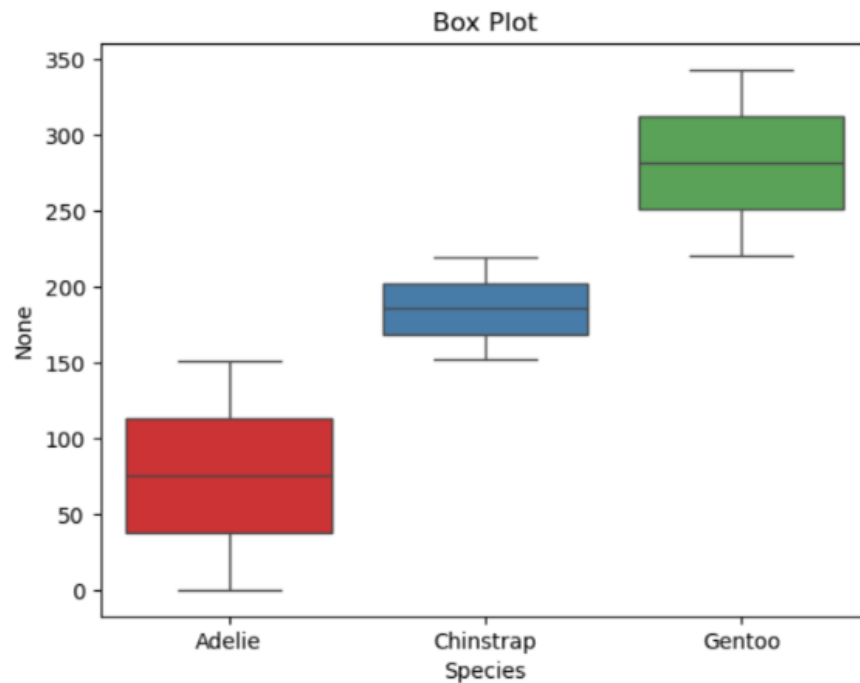
(344, 7)

[13]:

```
sns.histplot(ds,x="bill_length_mm",bins=20,color="forestgreen")
plt.title("Histogram")
plt.xlabel("Bill Length(mm)")
plt.show()
```

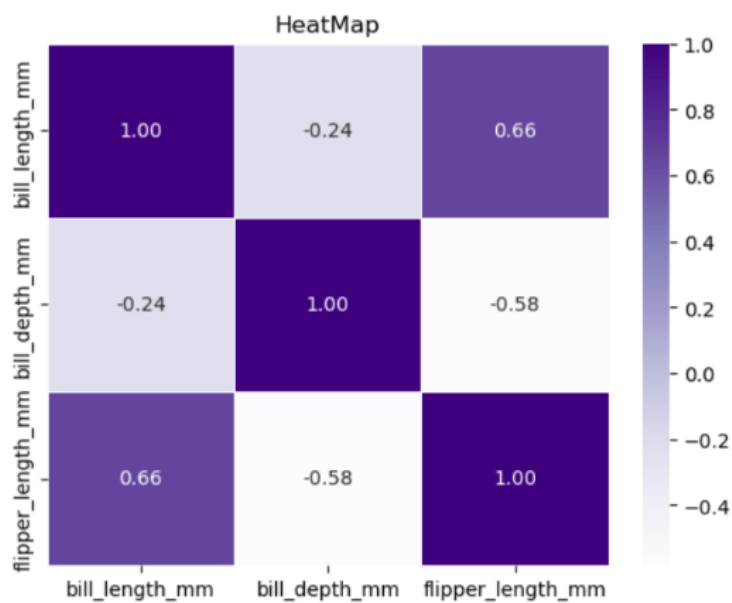


```
sns.boxplot(ds,x="species",y=ds.index,hue="species",palette="Set1")
plt.title("Box Plot")
plt.xlabel("Species")
plt.show()
```



[20]:

```
cols = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm"]
corr = ds[cols].corr()
sns.heatmap(corr,annot=True,cmap="Purples",fmt=".2f",linewidth=0.5)
plt.title("HeatMap")
plt.show()
```



3.2 Exploratory Data Analysis Techniques

Exploratory data analysis (EDA) techniques are methods used by data scientists to summarize dataset characteristics, detect outliers, and find hidden patterns using statistical summaries and visualizations.

1. Summary Statistics:

Use a sample dataset (e.g., the Iris dataset) to calculate and display summary statistics (mean, median, standard deviation, etc.).

[6]:

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris(as_frame=True)
df = iris.frame
print(df.head())
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	target
0	0
1	0
2	0
3	0
4	0

```
df = df.drop(columns=["target"])
print("First five rows:")
print(df.head())
```

First five rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
[101]
print("\nMean:")
print(df.mean())
print("\nMedian:")
print(df.median())
print("\nStandard Deviation:")
print(df.std())
print("\nMinimum:")
print(df.min())
print("\nMaximum:")
print(df.max())
print("\nVariance:")
print(df.var())
print("\nQuartiles:")
print(df.quantile([0.25, 0.50, 0.75]))
```

Mean:

sepal length (cm)	5.843333
sepal width (cm)	3.057333
petal length (cm)	3.758000
petal width (cm)	1.199333

dtype: float64

Median:

sepal length (cm)	5.80
sepal width (cm)	3.00
petal length (cm)	4.35
petal width (cm)	1.30

dtype: float64

Standard Deviation:

sepal length (cm)	0.828066
sepal width (cm)	0.435866
petal length (cm)	1.765298
petal width (cm)	0.762238

dtype: float64

Minimum:
sepal length (cm) 4.3
sepal width (cm) 2.0
petal length (cm) 1.0
petal width (cm) 0.1
dtype: float64

Maximum:
sepal length (cm) 7.9
sepal width (cm) 4.4
petal length (cm) 6.9
petal width (cm) 2.5
dtype: float64

Variance:
sepal length (cm) 0.685694
sepal width (cm) 0.189979
petal length (cm) 3.116278
petal width (cm) 0.581006
dtype: float64

Quartiles:
sepal length (cm) sepal width (cm) petal length (cm) petal width (cm)
0.25 5.1 2.8 1.60 0.3
0.50 5.8 3.0 4.35 1.3
0.75 6.4 3.3 5.10 1.8

```
print("\nComplete Summary Statistics:")  
print(df.describe())
```

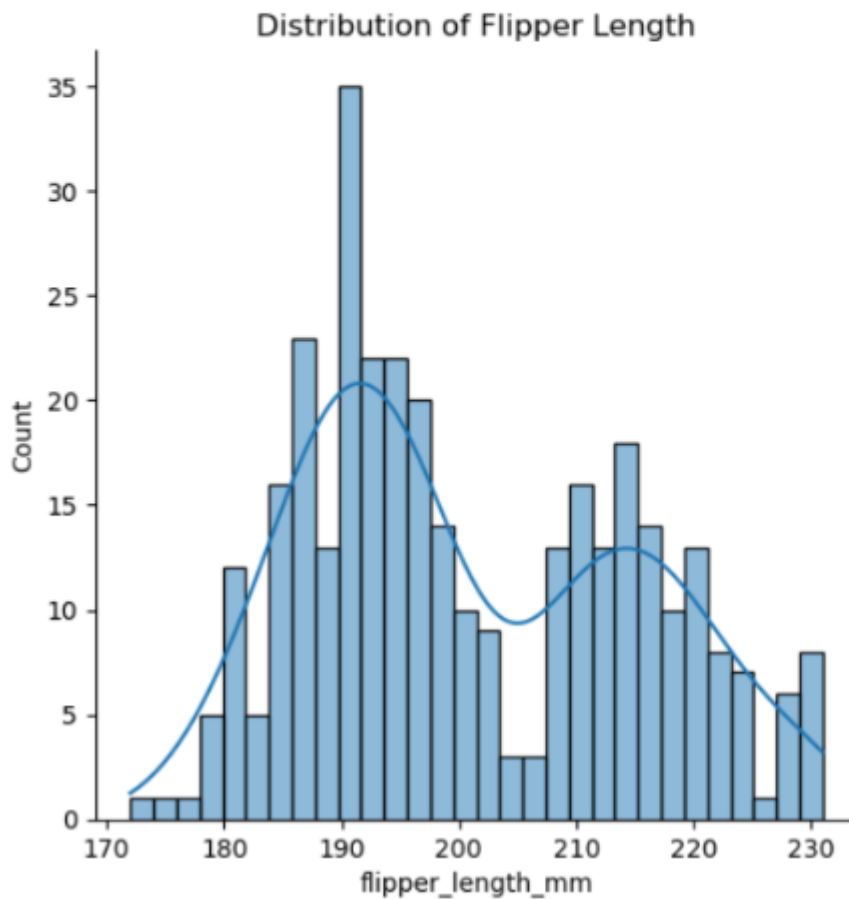
Complete Summary Statistics:

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
count	150.000000	150.000000	150.000000	
mean	5.843333	3.057333	3.758000	
std	0.828066	0.435866	1.765298	
min	4.300000	2.000000	1.000000	
25%	5.100000	2.800000	1.600000	
50%	5.800000	3.000000	4.350000	
75%	6.400000	3.300000	5.100000	
max	7.900000	4.400000	6.900000	

2. Data Distribution Analysis:

Use Seaborn to visualize the distribution of data for different variables using histograms and density plots.

```
import matplotlib.pyplot as plt
import seaborn as sns
data = sns.load_dataset("penguins")
sns.displot(data=data, x="flipper_length_mm", kde=True, bins=30)
plt.title("Distribution of Flipper Length")
plt.show()
```



3. Correlation Analysis:

Use a sample dataset to calculate the correlation matrix and visualize it using a heatmap.

```
from sklearn.datasets import fetch_california_housing
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns=housing.feature_names)
correlation_matrix = df.corr()
print("Correlation Matrix:")
print(correlation_matrix)
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="Blues"
)
plt.title("Correlation Matrix Heatmap")
plt.show()
```

Correlation Matrix:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
MedInc	1.000000	-0.119034	0.326895	-0.062040	0.004834	0.018766	-0.079809	-0.015176
HouseAge	-0.119034	1.000000	-0.153277	-0.077747	-0.296244	0.013191	0.011173	-0.108197
AveRooms	0.326895	-0.153277	1.000000	0.847621	-0.072213	-0.004852	0.106389	-0.027540
AveBedrms	-0.062040	-0.077747	0.847621	1.000000	-0.066197	-0.006181	0.069721	0.013344
Population	0.004834	-0.296244	-0.072213	-0.066197	1.000000	0.069863	-0.108785	0.099773
AveOccup	0.018766	0.013191	-0.004852	-0.006181	0.069863	1.000000	0.002366	0.002476
Latitude	-0.079809	0.011173	0.106389	0.069721	-0.108785	0.002366	1.000000	-0.002476
Longitude	-0.015176	-0.108197	-0.027540	0.013344	0.099773	0.002476	-0.002476	1.000000

